

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Факультет Компьютерных наук

Кафедра программирования и информационных технологий

Мобильное приложение для организации путешествий „Путевод“

Курсовая работа

Направление: 09.03.04. Программная инженерия

Зав. Кафедрой _____ д. ф.-м. н, доцент С.Д. Махортов

Руководитель _____ ст. преподаватель В.С. Тарасов

Руководитель практики _____ А.В. Москаленко

Обучающийся _____ Д.И. Шульженко, 3 курс, д/о

Обучающийся _____ А.Д. Едрышов, 3 курс, д/о

Обучающийся _____ М.Д. Ткачук, 3 курс, д/о

Обучающийся _____ И.С. Сторожев, 3 курс, д/о

Воронеж 2025

Содержание

Определения, обозначения и сокращения	4
Введение.....	8
1 Постановка задачи.....	10
1.1 Цели создания проекта	10
1.2 Функциональные требования к разрабатываемой системе	10
2 Анализ предметной области	12
2.1 Терминология (гlossарий) предметной области\.....	12
2.2 Анализ рынка путешествий России	12
2.3 Обзор аналогов.....	14
2.3.1 Copilot2Trip	15
2.3.2 TripIt	15
2.3.3 Tripomatic.....	16
2.4 Моделирование системы.....	17
2.4.1 Диаграммы Use-Case	17
2.4.2 Диаграммы активности	18
2.4.3 ER-диаграмма.....	18
3 Реализация.....	19
3.1 Средства реализации	19
3.1.1 Flutter.....	19
3.1.2 Java и Spring Boot.....	19
3.1.3 PostgreSQL.....	19
3.1.4 Docker.....	20
3.1.5 Дополнительные инструменты и технологии.....	20
3.2 Архитектурные особенности	20
3.3 Логика приложения	22
3.4 Серверная часть.....	22
3.4.1 Общая структура системы	22
3.4.2 База данных	23
3.4.3 CI/CD.....	23
3.4.4 Микросервис “External”	27
3.4.5 Микросервис “Library”.....	27
3.4.6 Микросервис “Planner”.....	28

3.4.7 Микросервис “Auth”	28
3.4.8 Общие интеграции.....	29
3.5 Клиентская часть.....	31
3.5.1 Архитектура мобильного приложения.....	31
3.5.2 Структура мобильного приложения	33
3.6 Реализация интерфейса	34
3.6.1 Реализация интерфейса мобильного приложения.....	34
3.6.2 Реализация интерфейса сайта для администрирования библиотеки маршрутов	46
4 Тестирование	48
5 Аналитика	49
Заключение	50
Список использованных источников	51

Определения, обозначения и сокращения

Терминология, используемая в курсовой работе представлена в таблице.

Таблица 1 — Терминология, используемая в курсовом проекте

Термин	Значение
Android	Операционная система с открытым исходным кодом, созданная для мобильных устройств.
ChangeNotifier	Базовый класс Flutter для оповещения слушателей об изменениях состояния.
CI/CD	Непрерывная интеграция (Continuous Integration) и доставка/деплой (Continuous Delivery/Deployment).
Circuit Breaker	Шаблон устойчивости, разрывающий цепочку вызовов к сбоящему сервису.
Dart	Язык программирования, используемый для разработки мобильных приложений на платформе Flutter.
Docker	Платформа для создания, распространения и запуска приложений в контейнерах.
Docker Swarm	Встроенный в Docker оркестратор контейнеров (альтернатива Kubernetes для малых кластеров).
Domain Driven Design (DDD)	Подход к проектированию, концентрирующийся на предметной области и её модели.
Draw.io	Онлайн-редактор для создания диаграмм и схем, используемый для разработки архитектуры системы и документации.
Figma	Инструмент для дизайна пользовательских интерфейсов, используемый для создания макетов и прототипов приложения.
Flutter SDK	Инструментальный набор для разработки мобильных приложений, использующий язык Dart для кросс-платформенной разработки.

Git/GitHub	Система контроля версий и платформа для хостинга кода, используемая для управления проектом и совместной работы.
Java	Язык программирования, используемый для разработки серверной части приложения.
JavaScript	Язык программирования, используемый для разработки веб-приложений, включая интерфейс администратора.
JSON	Формат данных для обмена информацией между компонентами системы.
JWT (JSON Web Token)	Стандарт аутентификации и авторизации с использованием токенов.
Liquibase	Инструмент для управления версиями схемы базы данных и автоматизации миграций.
MapStruct	Генератор Java-кода, создающий мапперы для преобразования DTO в Entity и обратно “на лету”.
Maven	Система сборки и управления зависимостями для проектов на Java/JVM.
Micrometer	Универсальная (facade) библиотека метрик для JVM-приложений, поддерживающая Prometheus, InfluxDB и др.
MVC	Шаблон “Model – View – Controller” — разделение данных, интерфейса и логики управления.
MVVM	Паттерн “Model – View – ViewModel”, популярный в UI-фреймворках.
NGINX	Высокопроизводительный веб-сервер и reverse-проxy, часто используемый как балансировщик нагрузки.
OpenTripMap API	

	Открытый API с данными о достопримечательностях, категориях и координатах POI.
PostgreSQL	Объектно-реляционная СУБД с поддержкой расширений (PostGIS и др.).
Postman	Графический клиент для тестирования и документации REST/HTTP-API.
Provider (Flutter)	Лёгкая библиотека управления состоянием приложений Flutter, основанная на InheritedWidget.
Retry-механизм	Автоматическое повторение операции при ошибке с контролируемыми интервалами/ограничениями.
Rolling Updates	Пошаговое обновление экземпляров сервиса без полной остановки кластера.
Spring Boot	Фреймворк, упрощающий создание Java-микросервисов по принципу “convention over configuration”.
Spring Security	Модуль Spring-экосистемы для аутентификации, авторизации и защиты веб-ресурсов.
Stateless-архитектура	Сервер не хранит состояния сессии; каждая HTTP-запрос обрабатывается изолированно.
Unsplash API	API фотостока Unsplash для получения лицензионных изображений высокой чёткости.
Yandex Geocoder	Сервис прямого и обратного геокодирования (адрес $\leftrightarrow$ координаты).
Yandex GeoSuggest	API автодополнения адресов/топонимов по частичному вводу пользователя.
Yandex GPT	Большая языковая модель Яндекса для генерации и понимания текста.
Zero-Downtime Deployment	Стратегия деплоя, гарантирующая непрерывную доступность приложения для пользователей.

Автодополнение	Подстановка вариантов (адресов, мест, тегов) по частичному вводу текста.
Администратор	Пользователь с расширенными правами доступа, который управляет учетными записями пользователей и модерацией контента.
Библиотека маршрутов	Публичная база данных маршрутов, где пользователи могут делиться и копировать маршруты.
Геокодирование	Преобразование адреса в координаты и обратное (reverse geocoding).
Интеграционное тестирование	Тестирование взаимодействия между компонентами системы.
Интерактивная карта	Карта, отображающая места и маршруты с возможностью редактирования и настройки отображения.
Клиент-серверная архитектура	Модель взаимодействия в сети, где клиент запрашивает ресурсы, а сервер их предоставляет.
Микросервисная архитектура	Архитектурный стиль разработки ПО, при котором приложение состоит из небольших независимых сервисов, взаимодействующих через API
Оффлайн-режим	Режим работы приложения без подключения к интернету, с последующей синхронизацией данных.
Фреймворк	Программная платформа, предоставляющая базовую структуру и набор инструментов для разработки приложений, что ускоряет создание кода и стандартизирует архитектуру.

Введение

Современное общество характеризуется активным развитием цифровых технологий, которые существенно изменяют способы организации различных аспектов повседневной жизни, в том числе планирования путешествий. Рост мобильности населения и доступности путешествий приводит к увеличению числа людей, стремящихся эффективно организовать свои поездки с помощью специализированных цифровых инструментов.

В условиях высокой конкуренции на рынке туристических сервисов пользователи сталкиваются с рядом проблем: необходимостью тщательно продумывать маршруты, учитывать временные ограничения, контролировать выполнение сопутствующих задач и обмениваться информацией с другими участниками путешествия. Одновременно с этим современные технологии позволяют автоматизировать многие из перечисленных процессов, сокращая временные и организационные издержки.

Целью данного проекта является разработка мобильного приложения «Путевод», предназначенного для автоматизации процесса планирования и организации путешествий. Ожидается, что внедрение подобного инструмента позволит повысить эффективность подготовки поездок, предоставить пользователям удобные средства для создания маршрутов, управления задачами, совместного редактирования планов и хранения воспоминаний о путешествиях.

Для достижения поставленных целей в работе будут проанализированы существующие решения на рынке, выявлены их сильные и слабые стороны, а также определены ключевые функции, наиболее востребованные конечными пользователями. На основе проведённого анализа будет предложен интуитивно понятный интерфейс, ориентированный на широкую аудиторию путешественников, а также реализованы уникальные возможности, такие как оффлайн-режим, публичная библиотека маршрутов и инструменты совместного планирования.

Реализация данного проекта призвана внести вклад в развитие цифровых сервисов в туристической отрасли, повысить качество пользовательского опыта и упростить процесс организации путешествий для различных категорий пользователей.

1 Постановка задачи

1.1 Цели создания проекта

Целью данного проекта является разработка мобильного приложения «Путевод» для организации и планирования путешествий, которое позволит пользователям эффективно создавать маршруты, управлять планами поездок и обмениваться ими с другими путешественниками.

Данное приложение разрабатывается с целью:

- с июня по август 2025 года привлечь не менее 60 активных пользователей;
- обеспечить публикацию не менее 30 уникальных маршрутов в публичной библиотеке в течение первых шести месяцев использования;
- достичь среднего уровня удовлетворённости пользователей функционалом приложения не ниже 8 баллов из 10 по результатам опроса тестовой группы;
- обеспечить, чтобы не менее 60% пользователей использовали функцию совместного планирования поездок хотя бы один раз за первые три месяца после регистрации;

1.2 Функциональные требования к разрабатываемой системе

Приложение должно обладать следующей функциональностью:

- создание и управление персональными планами поездок с возможностью почасовой детализации;
- просмотр и редактирование маршрутов на интерактивной карте с поддержкой поиска и добавления новых мест;
- формирование и ведение индивидуальных TODO-списков для каждой поездки, включая использование шаблонов и генерацию с помощью ИИ;
- публикация маршрутов в публичной библиотеке с возможностью копирования и оценки других маршрутов;

— совместное планирование и редактирование маршрутов с другими пользователями в режиме реального времени;

2 Анализ предметной области

2.1 Терминология (гlossарий) предметной области

Поездка (Trip) — совокупность дат, страны (городов) и плана мероприятий, объединённых одной целью путешествия.

День поездки (Trip Day) — календарный день, входящий в поездку, содержащий список событий и маршрут на карте.

Событие (Event) — конкретная активность в рамках дня: посещение места, переезд, отдых, отметка времени.

Маршрут — упорядоченная последовательность геоданных (точек/мест) для перемещения в ходе поездки.

Публичный маршрут — маршрут, опубликованный в библиотеке и доступный для просмотра/копирования другими пользователями.

Библиотека маршрутов — центральный каталог публичных маршрутов с поиском, фильтрацией, рейтингами и отзывами.

Совместное планирование — режим, позволяющий нескольким пользователям одновременно редактировать один маршрут.

TODO-список — список подготовительных или текущих задач, связанных с поездкой, отмечаемых как выполненные.

Онбординг — набор экранов или шагов, знакомящих нового пользователя с основными функциями приложения.

Оффлайн-режим — работа клиента без доступа к Интернету с последующей синхронизацией данных при появлении сети.

2.2 Анализ рынка путешествий России

По оценке BusinesStat, в 2024 г. россияне совершили **119 млн внутренних туристических поездок**, что на **10 %** больше, чем годом ранее — это самый высокий показатель за последние пять лет [1] (см. рисунок 1).



Рисунок 1 — Динамика внутреннего туристического потока в России, 2020-2024 гг.

Одновременно наблюдается устойчивый рост числа самостоятельных путешественников. По данным НАФИ, доля россиян, предпочитающих самостоятельно планировать поездки без помощи туроператоров, за последние пять лет увеличилась с 37% до 48% [2] (см. рисунок 2).



Рисунок 2 — Доля самостоятельных путешественников в России, 2019–2024 гг.

Эксперты отмечают, что такими темпами рынок самостоятельных путешествий будет продолжать расти благодаря цифровизации, развитию

онлайн-сервисов и увеличению доступности мобильных приложений для планирования маршрутов. Кроме того, россияне всё чаще отдают предпочтение автотуризму: согласно данным, ВЦИОМ, в 2024 году 32% граждан совершали поездки на собственном транспорте или в арендованных автомобилях [3].

Таким образом, развитие цифровых платформ, помогающих в самостоятельном планировании путешествий, отвечает актуальным тенденциям рынка и формирующимся потребностям аудитории.

2.3

2.4 Обзор аналогов

В ходе исследования рынка мобильных приложений, предназначенных для планирования и организации путешествий, были выявлены три основных прямых конкурента. В таблице 2 приведены результаты проведённого анализа аналогичных решений, отражающие их ключевые функции и отличия от разрабатываемого приложения.

Таблица 2 — Результаты конкурентного исследования

Функциональность	Copilot2Trip	Tripit	Sygyic Travel
Карта с местами путешествия	+	-	+
Инструмент для заметок, TODO листов	-	+	-
Возможность поделиться маршрутами	+	-	+
Библиотека маршрутов	-	-	-
Офлайн-доступ	-	-	+
ИИ для создания TODO-листов	+	-	-
ИИ для создания маршрутов	+	-	-

2.4.1 Copilot2Trip

Copilot2Trip позиционируется как «AI-powered personal travel assistant» [4]. Сервис формирует персонализированные маршруты, показывает их на интерактивной карте, предлагает рекомендации по достопримечательностям и умеет генерировать план поездки по заданным предпочтениям. По сути, вы общаетесь с ИИ-помощником, который делает всё за вас, вам лишь нужно правильно составить обращения. Процесс общения и составления маршрута с ИИ-агентом вы можете увидеть на рисунке 3.

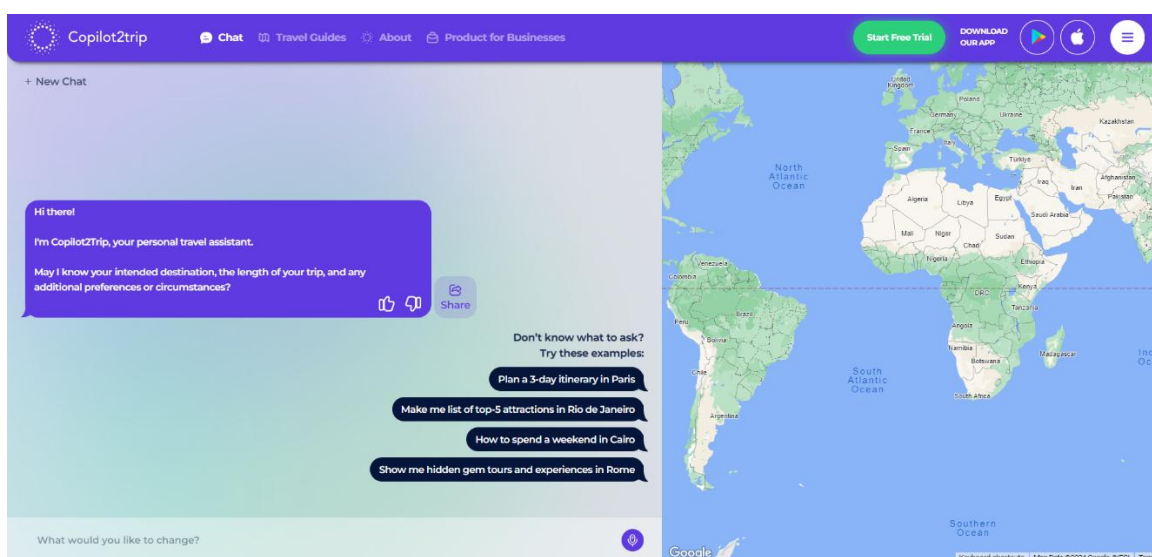


Рисунок 3 — Интерфейс создания маршрута в Copilot2Trip

Пользователь может делиться итоговым маршрутом через ссылку и редактировать его в диалоговом интерфейсе. Однако приложение не поддерживает офлайн-режим и не содержит встроенных TODO-листов.

2.4.2 TripIt

TripIt — один из самых узнаваемых агрегаторов бронирований. Приложение автоматически собирает письма-подтверждения о перелётах, отелях и аренде авто, формируя единый интерактивный график путешествия. Пользователь может добавлять заметки и задачи к каждому элементу маршрута. Часть данных кэшируется на устройстве и остаётся доступной без подключения к интернету, но карта и редактирование офлайн недоступны. Рисунок 4 демонстрирует интерфейс TripIt.



Рисунок 4 — Интерфейс TripIt

Совместная работа реализована через приглашения по e-mail, без полноценного совместного редактирования. Расширенные функции (оповещения об изменениях рейса, компенсации) доступны в платной подписке TripIt Pro.

2.4.3 Tripomatic

Tripomatic строит подробные путевые планы с визуализацией на офлайн-картах. Данные карты хранятся локально и регулярно обновляются. Пользователь может планировать день-за-днём, перетаскивая места интереса, и делиться готовым маршрутом. Сильной стороной сервиса остаётся навигация с голосовыми подсказками и поддержка автономного режима. При этом в приложении отсутствуют встроенные TODO-списки и инструменты искусственного интеллекта для автоматического планирования. На рисунке 6 показан процесс создания путешествия в сервисе.

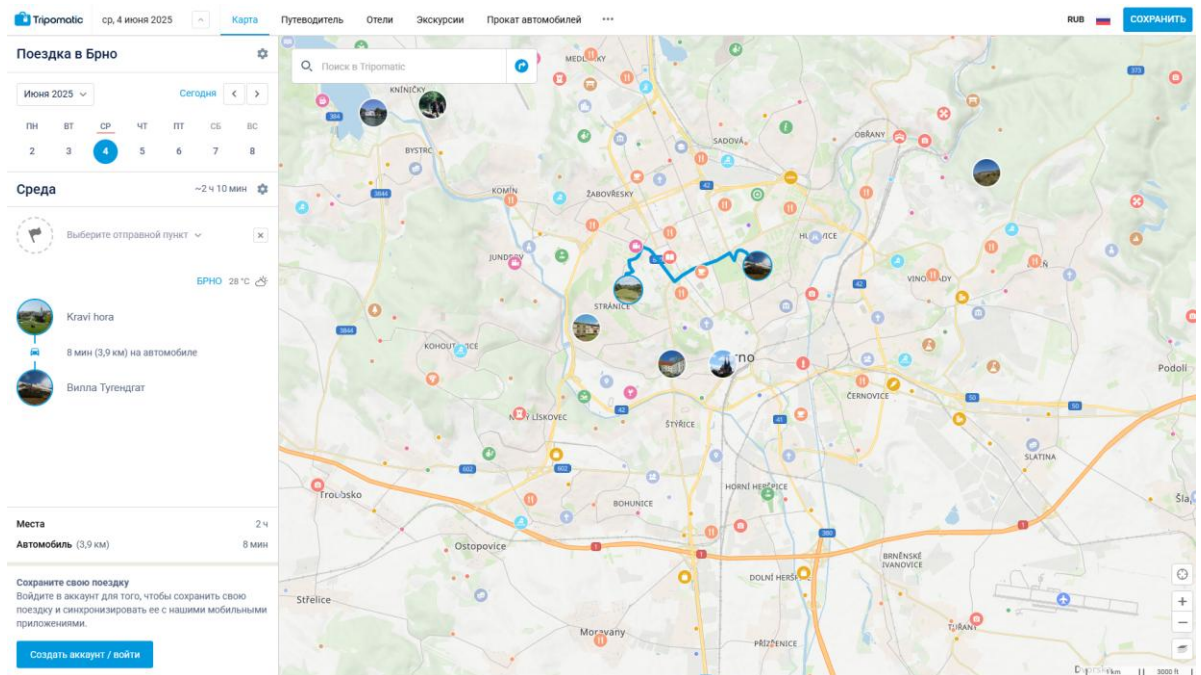


Рисунок 5 — Создание путешествия в TripIt

2.5 Моделирование системы

2.5.1 Диаграммы Use-Case

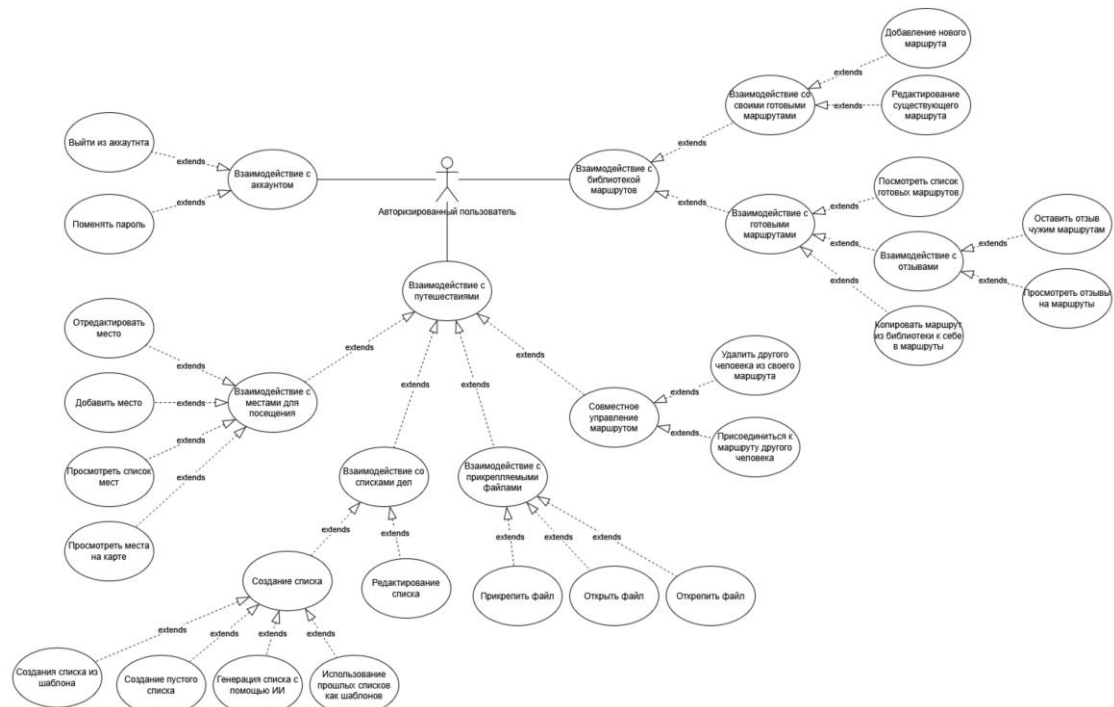


Рисунок 6 — Диаграмма Use Case авторизованного пользователя приложения

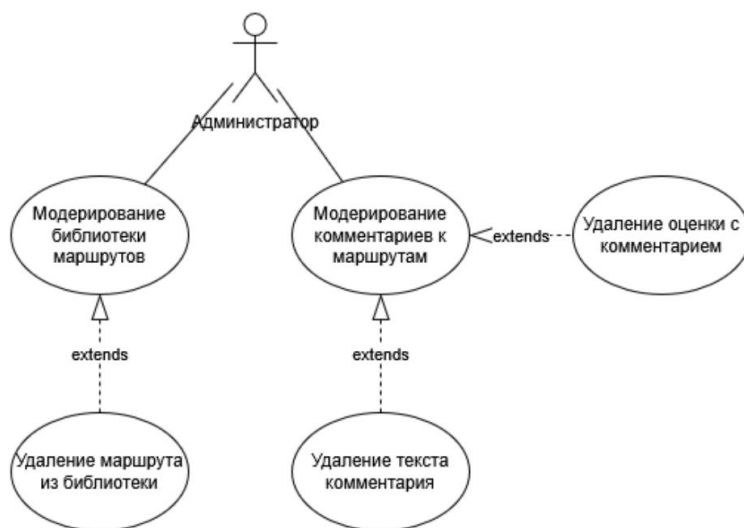


Рисунок 7 — Диаграмма Use Case администратора в админ-панели

2.5.2 Диаграммы активности

Диаграммы активности представлены в Приложении Б рисунками Б.1 и Б.2.

2.5.3 ER-диаграмма

Диаграмма со схемой базы данных представлена в Приложении А рисунком А.1.

3 Реализация

3.1 Средства реализации

3.1.1 Flutter

Для реализации клиентской части мобильного приложения использовались язык программирования Dart версии 3.3.0 и Flutter SDK версии 3.19.1. Flutter — это современный кроссплатформенный фреймворк, позволяющий разрабатывать приложения для Android, iOS, web и desktop на единой кодовой базе [4].

Использование Flutter обеспечивает следующие преимущества:

- Быстрое создание пользовательских интерфейсов за счёт гибкой системы виджетов и развитого инструментального набора;
- Высокая производительность приложений благодаря прямой компиляции в нативный код;
- Развита экосистема библиотек и плагинов, что упрощает интеграцию с картографическими сервисами, работу с сетью, хранение и синхронизацию данных, а также реализацию push-уведомлений;
- Удобство поддержки и масштабирования за счёт единого стека технологий и стандартизированных подходов.

3.1.2 Java и Spring Boot

Серверная часть приложения реализована с использованием языка программирования Java версии 23 и фреймворка Spring Boot 3.2.3 [5]. Java была выбрана в качестве основного языка благодаря своей высокой производительности, стабильности и широкой поддержке в индустрии. Spring Boot обеспечивает ускоренную разработку серверных микросервисов и поддержку REST API, необходимых для взаимодействия с мобильным клиентом.

3.1.3 PostgreSQL

В качестве системы управления базами данных используется PostgreSQL версии 16.2. Данная реляционная СУБД отличается надёжностью,

масштабируемостью и поддержкой сложных операций с данными. PostgreSQL обеспечивает эффективное хранение информации о пользователях, маршрутах, событиях, списках задач и других сущностях приложения.

3.1.4 Docker

Для развёртывания серверных компонентов системы применяется платформа контейнеризации Docker. Использование Docker позволяет создавать изолированные контейнеры для различных сервисов, обеспечивая переносимость между средами разработки и продакшен-средой, а также упрощая процесс масштабирования. Организация совместной работы сервисов обеспечивается с помощью Docker Compose.

3.1.5 Дополнительные инструменты и технологии

В процессе разработки и сопровождения проекта используются следующие средства:

- **Git и GitHub** — для контроля версий, совместной работы над исходным кодом и управления релизами;
- **Figma** — для проектирования пользовательского интерфейса и создания макетов;
- **Draw.io и Miro** — для построения архитектурных схем, UML- и ER-диаграмм;
- **YouGile** — для управления задачами и организации работы команды;
- **JWT (JSON Web Token)** — для аутентификации и авторизации пользователей;
- **Postman** — для тестирования REST API.

3.2 Архитектурные особенности

Архитектура приложения соответствует трёхзвенной клиент-серверной модели. Взаимодействие между клиентской и серверной частями реализовано через REST API с использованием защищённого протокола HTTPS и формата обмена данными JSON. На рисунке 8 вы можете видеть схему архитектуры.

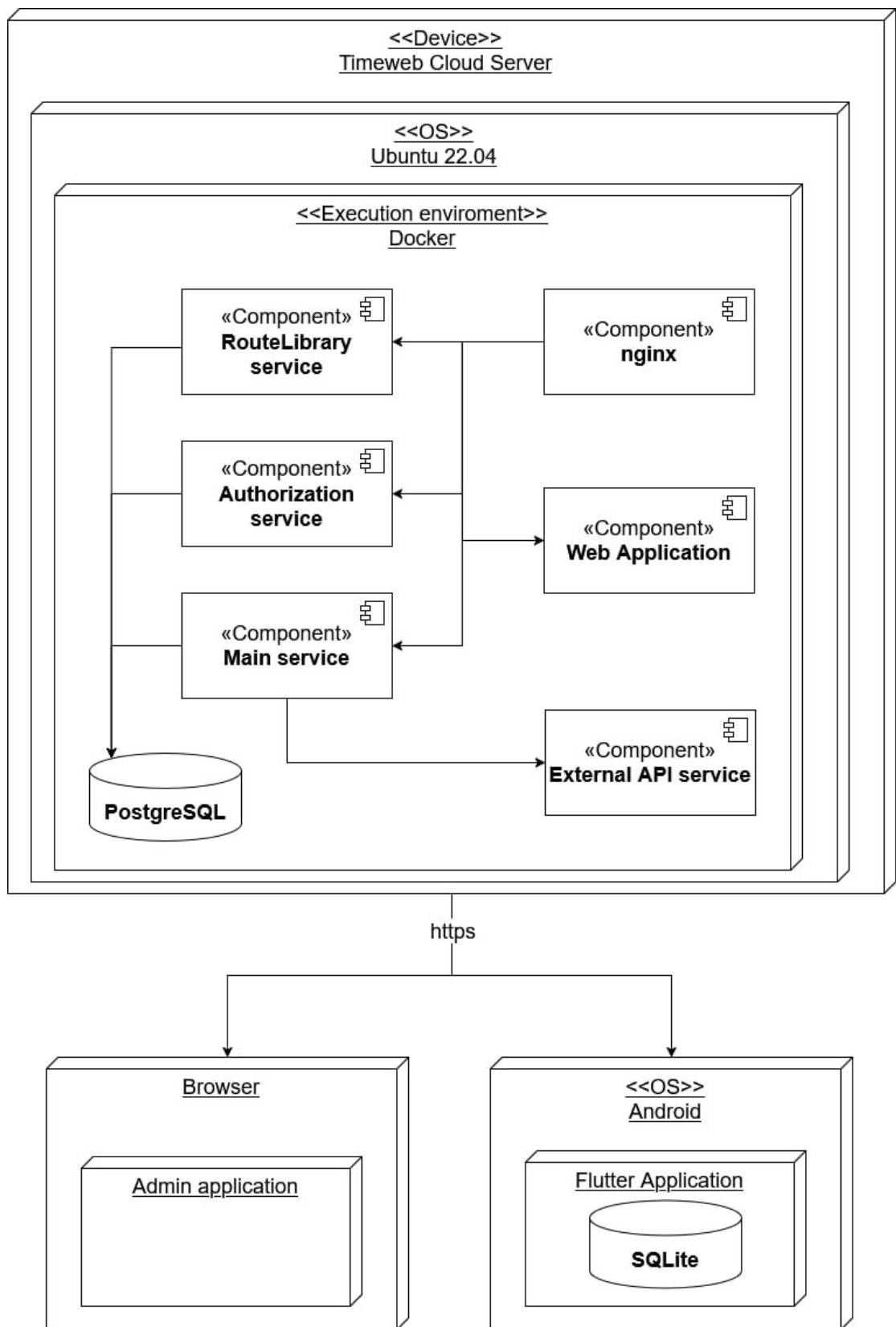


Рисунок 8 — Диаграмма развёртывания

В архитектуре приложения NGINX выполняет роль обратного прокси-сервера: принимает HTTP-запросы и перенаправляет их на бэкенд. В свою

очередь бэкенд, состоящий из четырёх микросервисов, обрабатывает запросы, взаимодействует с СУБД PostgreSQL, а также взаимодействует с внешними API.

Для обеспечения оффлайн-режима мобильное приложение использует локальное кэширование данных с последующей синхронизацией при восстановлении подключения к сети.

3.3 Логика приложения

В системе пользователь имеет одну из ролей:

- неавторизованный пользователь;
- авторизованный пользователь;
- администратор приложения.

3.4 Серверная часть

3.4.1 Общая структура системы

Серверная часть приложения «Путевод» реализована на Java 23 с использованием Spring Boot 3.2.3. Она инкапсулирует всю бизнес-логику: от аутентификации и синхронизации данных до интеграции с картографическими сервисами и отправки push-уведомлений. Каждый микросервис построен по MVC (Model-View-Controller) архитектуре с дополнительными слоями для обеспечения разделения ответственности и maintainability кода.

Система состоит из четырех основных микросервисов, каждый из которых отвечает за определенную функциональность:

- Auth Service - сервис аутентификации и авторизации
- Planner Service - сервис планирования маршрутов
- External Service - сервис интеграции с внешними API
- Library Service - сервис библиотеки публичных маршрутов

Архитектура построена по принципам Domain Driven Design с четким разделением ответственности между сервисами. Для управления

зависимостями используется Maven, структура проекта организована как многомодульный Maven-проект с родительским POM-файлом.

3.4.2 База данных

Система использует PostgreSQL 17 в качестве основной СУБД. Для управления миграциями базы данных применяется Liquibase 4.31.1, что обеспечивает версионирование схемы и безопасное развертывание изменений.

Структура миграций:

- 00-initial-schema.sql - начальная схема базы данных
- 01-add-trip-preview.sql - добавление превью для поездок
- 02-combine-ratings-and-comments.sql - объединение рейтингов и комментариев
- 003_anonymous_users.sql - поддержка анонимных пользователей
- 04-add-preview-url-fields.sql - добавление полей для URL превью

База данных содержит таблицы для пользователей, поездок, дней поездок, событий, доступов к поездкам, опубликованных маршрутов, оценок и других сущностей системы. Подробную ER-диаграмму базы данных вы можете увидеть на рисунке А.1.

Все микросервисы используют единую базу данных, что упрощает транзакционность операций между сервисами.

3.4.3 CI/CD

Система непрерывной интеграции и доставки реализована с использованием GitHub Actions и Docker Swarm для оркестрации контейнеров в production среде:

CI-процесс для проекта «Путевод» реализован с помощью пайплайна ci.yml, который запускается автоматически при каждом push и создании pull request. В рамках CI-процедуры используется Java 23 (дистрибутив Temurin), что обеспечивает актуальность и совместимость используемых технологий.

Перед запуском тестов автоматически выполняются миграции базы данных с помощью Liquibase, благодаря чему структура тестовой базы всегда соответствует текущему состоянию проекта. Далее для каждого микросервиса

выполняются компиляция, тестирование и упаковка, что позволяет выявлять ошибки на ранних этапах разработки. Все собранные артефакты (JAR-файлы) сохраняются и могут быть использованы для дальнейших этапов доставки и развертывания.

В процессе тестирования CI-пайплайна осуществляется интеграция с контейнеризированной PostgreSQL, что позволяет проверять работоспособность сервисов в условиях, максимально приближенных к реальному окружению. Такой подход обеспечивает стабильность, предсказуемость и повторяемость всех операций, связанных со сборкой и тестированием микросервисов приложения «Путевод».

Сам же пайплайн деплоймента приложения «Путевод» реализован в виде автоматизированного сценария (`deployment_pipeline.yml`), который включает несколько ключевых этапов. На первом этапе, при каждом `push` в ветку `develop`, происходит автоматический запуск пайплайна, сопровождаемый параллельным выполнением юнит- и интеграционных тестов всех микросервисов в изолированной среде с использованием контейнеризированной PostgreSQL. Таким образом, все сервисы проходят валидацию на предмет работоспособности и соответствия требованиям до перехода к следующему этапу.

На этапе сборки создаются отдельные Docker-образы для каждого микросервиса: `putevod-auth`, `putevod-planner`, `putevod-external` и `putevod-library`. Для хранения образов используется GitHub Container Registry (`ghcr.io`), что обеспечивает централизованный и безопасный доступ к актуальным версиям. Все образы дополнительно тегируются по хешу коммита, что позволяет обеспечить прозрачное версионирование и быстрый откат к нужной версии в случае необходимости. Сборка всех сервисов осуществляется параллельно для ускорения процесса доставки изменений.

Этап развертывания включает автоматическую генерацию файла окружения с необходимыми секретами, после чего производится деплой стека сервисов с помощью Docker Swarm и команды `docker stack deploy`.

Подключение к production серверу осуществляется по защищённому SSH-каналу с использованием приватного ключа. Для скачивания Docker-образов реализована аутентификация в Container Registry. Такой подход обеспечивает надёжность, безопасность и воспроизводимость процесса развертывания приложения на целевой инфраструктуре.

При помощи Docker Swarm оркестрация сервисов в системе «Путевод» организована с использованием PostgreSQL 17 в качестве основной базы данных с надёжным персистентным хранилищем. Для автоматического применения всех необходимых миграций структуры данных при развертывании используется инструмент Liquibase, что позволяет поддерживать актуальное состояние схемы базы данных на всех этапах жизненного цикла приложения.

В инфраструктуре развернуты четыре микросервиса, между которыми выстроены чёткие зависимости. Оркестрация обеспечивает корректный порядок запуска сервисов: база данных и миграции иницируются в первую очередь, затем последовательно стартуют все основные микросервисы, каждый из которых получает необходимые параметры подключения и настройки. Такой подход гарантирует согласованность работы всех компонентов и стабильную работу приложения как в процессе запуска, так и при обновлении.

В процессе развертывания серверной части приложения «Путевод» применяется стратегия обновления start-first, что позволяет обеспечивать zero-downtime deployment — обновление сервисов происходит без прерывания их работы для пользователей. Все сервисы и компоненты запускаются с учётом зависимостей, что гарантирует корректный порядок инициализации при старте или обновлении кластера.

Для внешнего доступа к сервисам реализован маппинг портов: Auth Service доступен на порту 8081, Planner Service — на 8082, External Service — на 8083, Library Service — на 8084, а база данных PostgreSQL — на 5438 порту. Управление конфигурацией осуществляется через внешний файл окружения,

который используется для безопасного хранения секретов и автоматической инъекции переменных при развертывании. Подключение всех сервисов к базе данных настраивается централизованно, что упрощает администрирование и снижает вероятность ошибок.

Использование Docker Swarm даёт приложению ряд преимуществ [6]. Во-первых, это высокая доступность — автоматический перезапуск контейнеров в случае сбоев и встроенный механизм балансировки нагрузки между экземплярами сервисов. Поддерживаются rolling updates с применением стратегии start-first, позволяющей обновлять сервисы без простоя. Система service discovery обеспечивает автоматическое обнаружение всех компонентов внутри кластера, а масштабирование сервисов возможно в горизонтальном режиме в зависимости от нагрузки. Дополнительно реализован постоянный мониторинг состояния контейнеров с автоматическим восстановлением в случае возникновения проблем.

Особое внимание уделяется вопросам безопасности. Доступ к production серверу осуществляется исключительно по SSH-ключам, а все секреты хранятся в системе GitHub Secrets. Для доступа к приватным образам в контейнерном реестре используется аутентификация, а сервисы изолируются в рамках overlay-сетей Docker Swarm.

Мониторинг процесса развертывания включает логирование всех этапов, автоматическую валидацию успешности обновления стека и возможность отката к предыдущей версии при возникновении ошибок. Кроме того, система интегрирована с инструментами мониторинга, что позволяет в режиме реального времени отслеживать состояние и производительность всех сервисов приложения.

Такая архитектура CI/CD обеспечивает надежное, автоматизированное и безопасное развертывание микросервисной системы с минимальными простоями и возможностью быстрого отката в случае проблем.

3.4.4 Микросервис “External”

Микросервис внешних интеграций (External Service) реализует взаимодействие приложения «Путевод» с рядом сторонних API для расширения функциональных возможностей. В частности, для поиска достопримечательностей и интересных мест используется OpenTripMap API, а для автодополнения и геокодирования адресов — сервисы Yandex GeoSuggest и Yandex Geocoder [7]. Генерация описаний поездок осуществляется с помощью Yandex GPT, а для получения изображений городов и достопримечательностей применяется Unsplash API [8].

Для организации асинхронных HTTP-запросов применяется WebClient, что позволяет оптимизировать задержки при обращении к внешним сервисам. В архитектуре предусмотрены fallback-механизмы на случай недоступности сторонних систем, а также кэширование ответов для повышения производительности и снижения количества обращений к API. Обработка ошибок и таймаутов реализована с учётом устойчивости системы.

Паттерны интеграции включают использование circuit breaker для отказоустойчивости, retry-механизмов при временных сбоях внешних сервисов, а также адаптеров, обеспечивающих унификацию ответов различных API и упрощающих работу с ними для основной бизнес-логики приложения

3.4.5 Микросервис “Library”

Микросервис библиотеки (Library Service) отвечает за публикацию и управление публичными маршрутами, а также обеспечивает доступ к системе оценок, отзывов и модерации контента. Пользователи могут не только создавать маршруты и делиться ими с другими, но и копировать понравившиеся маршруты в свои личные поездки. Реализованы функции поиска и фильтрации маршрутов по названию, стране, городу и длительности путешествия, а также сортировка по популярности и рейтингу. Система рекомендаций учитывает активность пользователя, что способствует персонализации выдачи.

3.4.6 Микросервис “Planner”

Микросервис планирования (Planner Service) реализует всю бизнес-логику, связанную с созданием, хранением и совместным управлением поездками. Пользователь может создавать новые поездки, планировать маршрут по дням с добавлением событий и мест, приглашать других пользователей для совместного редактирования, а также публиковать маршруты в публичную библиотеку. Для каждой поездки и отдельно предусмотрена система todo-списков, которые помогают пользователю организовать подготовку и выполнить все необходимые задачи.

В архитектуре Planner Service используются современные подходы: для маппинга между DTO и сущностями применяется MapStruct, для доступа к данным — Spring Data JPA с возможностью написания кастомных SQL-запросов. Реализована система уведомлений о приглашениях и изменениях в маршрутах. При создании поездки с указанием диапазона дат дни формируются автоматически, что ускоряет начальное планирование. Сервис поддерживает анонимных пользователей, что позволяет использовать базовые возможности приложения даже без регистрации.

Модель данных сервиса включает следующие основные сущности: Trip (поездка), TripDay (дни поездки), Event (события внутри дня), TripAccess (права доступа), а также TodoList (списки задач для поездки). Такой подход обеспечивает гибкость управления данными, поддерживает расширение функциональности и повышает удобство для конечных пользователей.

3.4.7 Микросервис “Auth”

Микросервис аутентификации (Auth Service) отвечает за регистрацию и авторизацию пользователей в системе «Путевод». Он реализует полную работу с JWT-токенами (access и refresh), поддерживает анонимный режим для гостей, а также обеспечивает верификацию email и восстановление паролей. Для межсервисного взаимодействия реализована система сервисных токенов, что позволяет безопасно интегрировать отдельные компоненты приложения.

В качестве технологической основы используется Spring Security, обеспечивающий защиту всех эндпоинтов. Для генерации и проверки JWT-токенов применяется библиотека jjwt, а для безопасного хранения паролей — алгоритм Bcrypt. Отправка писем для подтверждения регистрации и восстановления пароля реализована через Spring Mail. Вся документация API поддерживается и обновляется с помощью Swagger/OpenAPI, что упрощает тестирование и интеграцию. Безопасность приложения дополнительно обеспечивается двухуровневой системой токенов, отслеживанием сессий с привязкой к устройствам, поддержкой CORS для фронтенд-приложений, а также отдельными сервисными токенами для авторизации внутренних запросов между микросервисами.

3.4.8 Общие интеграции

Для мониторинга и анализа состояния системы применяются современные инструменты: сбор метрик осуществляется с помощью Prometheus, а их визуализация — через дашборды Grafana. Созданы специализированные панели мониторинга для анализа общей статистики системы, технических и пользовательских метрик, отслеживания состояния поездов и контроля за работой внешних интеграций. Здоровье сервисов контролируется через Spring Boot Actuator и Micrometer, что позволяет отслеживать производительность HTTP-запросов и бизнес-метрики в реальном времени.

В рамках обеспечения высокого качества кода используются инструменты для анализа покрытия тестами (JaCoCo), внедрены аннотации и аспекты для отслеживания метрик, реализована строгая валидация данных средствами Bean Validation. Для повышения производительности применяется ленивая загрузка связей в JPA, пагинация больших списков, оптимизированные SQL-запросы и сервисное кеширование.

Масштабируемость системы достигается за счёт stateless-архитектуры микросервисов, возможности горизонтального масштабирования с использованием Docker и асинхронной обработки ресурсоёмких операций.

Все эти меры обеспечивают высокую надёжность, производительность и удобство поддержки, позволяя эффективно развивать функционал приложения при росте пользовательской базы.

3.4.9 Использование YandexGPT

Одной из ключевых инноваций приложения «Путевод» стала интеграция с Yandex GPT API для автоматической генерации персонализированных списков вещей для путешествий на основе контекста поездки и пользовательских запросов.

ИИ-функциональность вынесена в отдельный микросервис External Service, что обеспечивает чёткое разделение ответственности. Главный сервис интеграции — YandexGptTripListService — реализует три режима генерации списков:

- По свободному тексту: Пользователь описывает свою поездку, и система формирует индивидуальный список вещей, предварительно проверяя корректность и безопасность запроса.

- По данным о конкретной поездке: Сервис получает детали маршрута через Planner Service и генерирует список с учётом всех параметров (даты, города, страна).

- На основе шаблона: Используются предустановленные шаблоны для разных типов путешествий, которые нейросеть дополняет с учётом контекста поездки.

Технически обмен с Yandex GPT построен на REST-архитектуре. Для каждого запроса формируется системный промпт с чёткой инструкцией и всеми параметрами маршрута, а пользовательское сообщение содержит сам запрос. В ответ нейросеть формирует только список без лишних пояснений. Для гибкости конфигурация API, ключи, параметры генерации и endpoint задаются через внешние переменные среды.

Безопасность интеграции строится на ограничении доступа к ИИ-функциям только для зарегистрированных пользователей и многоуровневой фильтрации пользовательских запросов на запрещённые темы. Все ответы

нейросети автоматически парсятся и преобразуются в структурированные списки для передачи в клиент.

Интеграция с микросервисами осуществляется через PlannerClient, который обеспечивает сквозную авторизацию и обработку ошибок при межсервисном обмене.

Вся работа с ИИ-функциями логируется и мониторится: фиксируются параметры запроса, производительность, ошибки и востребованность каждого режима. Для оптимизации расходов на AI-сервис установлен лимит на размер ответа, а доступ к функциям имеют только зарегистрированные пользователи.

Таким образом, интеграция Yandex GPT позволяет «Путеводу» предлагать пользователям интеллектуальную генерацию персональных списков вещей с учётом контекста, сохраняя при этом безопасность, производительность и гибкость архитектуры для будущего развития.

3.5 Клиентская часть

3.5.1 Архитектура мобильного приложения

Архитектура мобильного приложения «Путевод» построена на паттерне MVVM (Model-View-ViewModel) с применением пакета Provider для управления состоянием, что соответствует современным стандартам разработки на Flutter и обеспечивает удобное масштабирование и поддержку кода.

В качестве технологической основы используется Flutter SDK версии 3.2.3 и выше, а основным языком программирования выбран Dart. Для реализации некоторых функций используется нативный код Android. Это позволяет максимально использовать преимущества платформы и гибко интегрировать сторонние сервисы.

В визуальном слое (view) реализованы более 25 экранов с использованием Flutter Material Design. UI управляется реактивно через состояние в ViewModel, используя паттерн ChangeNotifier. Все

пользовательские действия передаются в ViewModel через методы, которые обновляют состояние и уведомляют UI об изменениях.

ViewModel служит промежуточным звеном между UI и внешними сервисами. Каждый экран имеет соответствующий ViewModel, который инкапсулирует бизнес-логику, управляет состоянием загрузки, обрабатывает ошибки и координирует взаимодействие с API. Состояние экрана включает данные поездок, TODO элементы, профиль пользователя и статус сетевых операций.

Внешние сервисы (external layer) реализованы как отдельные классы для работы с микросервисной архитектурой бэкенда: AuthService, TripService, LibraryService и ExternalService. Все HTTP-запросы осуществляются через универсальный ApiClient с автоматической обработкой JWT токенов, перехватом ошибок 401 и логированием запросов.

Микросервисная интеграция реализована через четыре специализированных API клиента: сервис авторизации (auth), планировщик поездок (planner), внешние API (external) и библиотека маршрутов (library). Каждый сервис инкапсулирует работу с соответствующим бэкенд-микросервисом и обрабатывает специфичные для домена операции.

Управление состоянием централизовано через Provider с MultiProvider в корне приложения, обеспечивая инъекцию всех ViewModel'ей и сервисов. Это гарантирует реактивные обновления UI, правильную утилизацию ресурсов и возможность доступа к общему состоянию из любой точки приложения.

Навигация реализована стандартными средствами Flutter с программными переходами между экранами. Приложение поддерживает полный цикл пользовательского пути: онбординг, авторизацию, основные функции планирования поездок, работу с AI для генерации TODO, интерактивные карты и социальные функции.

Таким образом, архитектура сочетает принципы разделения ответственности, реактивного программирования и микросервисной

интеграции, обеспечивая масштабируемость и удобство сопровождения проекта для планирования путешествий.

3.5.2 Структура мобильного приложения

Model (lib/model/):

В этом слое располагаются основные модели данных приложения, такие как `trip.dart`, `place.dart`, `todo_item_detail.dart`, а также сервисы (`notification_service.dart`, `auth_model.dart`) и конфигурационные файлы (`constants.dart`, `app_colors.dart`). Управление локальными настройками реализовано через менеджер `shared preferences` (`shared_prefs_manager.dart`).

View (lib/view/):

Этот слой отвечает за визуальное отображение информации и взаимодействие с пользователем. Здесь расположены экраны (`screens/`), которых в проекте более 25, а также переиспользуемые UI-компоненты (`widgets/`), обеспечивающие единый стиль и удобство разработки.

ViewModel (lib/view-model/):

Для каждого экрана разработан отдельный ViewModel-класс. Управление состоянием осуществляется по паттерну `ChangeNotifier` совместно с `Provider`. Среди примеров — `trip_creation_view_model.dart`, `profile_view_model.dart` и другие, что позволяет реализовать реактивный подход и отделить бизнес-логику от пользовательского интерфейса.

Внешние сервисы (lib/external/):

В отдельной директории реализованы модули для работы с внешними API и сервисами: модули аутентификации, управления поездками, библиотеки маршрутов, интеграций и отправки уведомлений.

Модульная архитектура API-клиентов облегчает добавление новых интеграций и поддержку существующих.

3.6 Реализация интерфейса

3.6.1 Реализация интерфейса мобильного приложения

Интерфейс мобильного приложения «Путевод» оформлен в едином стиле с использованием ограниченного набора шрифтов и современной цветовой палитры. Корректное отображение всех экранов обеспечено на устройствах с операционной системой Android 11 и выше.

При первом запуске приложения пользователю демонстрируется onboarding — серия информативных экранов, содержащих краткое описание основных возможностей и преимуществ «Путевода» (см. рисунок 9). Данный раздел знакомит пользователя с основными функциями приложения.

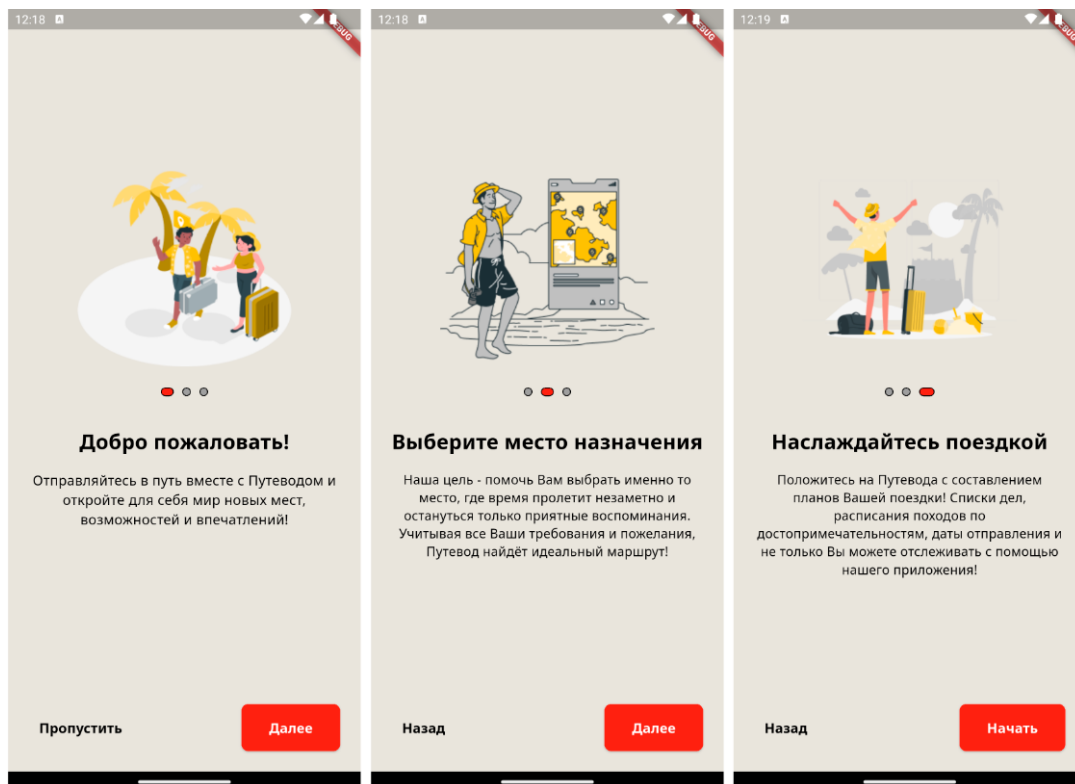


Рисунок 9 — Onboarding при первом запуске приложения

Далее перед пользователем появляется экран входа, где он может перейти либо к регистрации в приложении, либо же войти или продолжить как гость (см. рисунок 10).

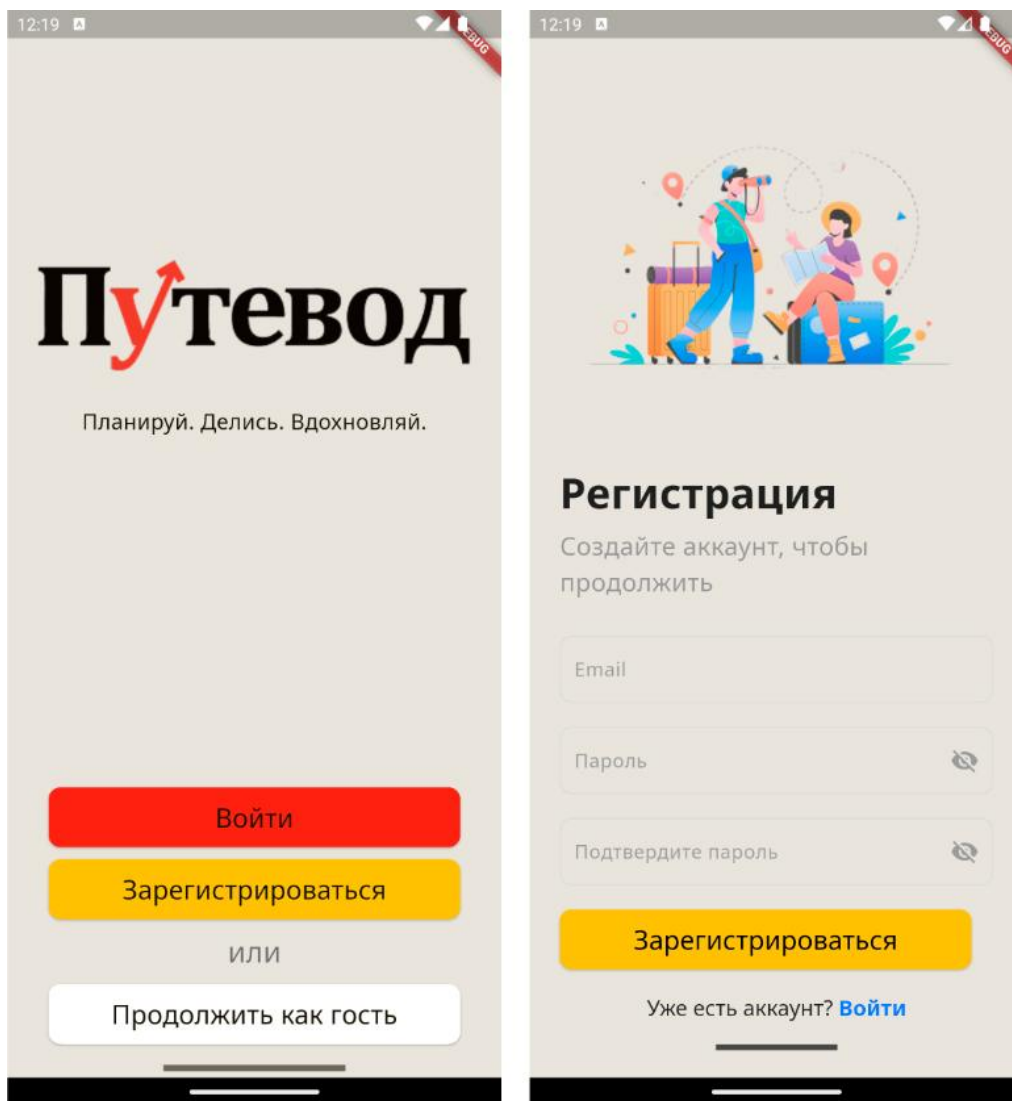


Рисунок 10 — Экран регистрации нового пользователя

Для входа в приложение используется стандартная форма авторизации с поддержкой восстановления пароля (см. рисунок 11).

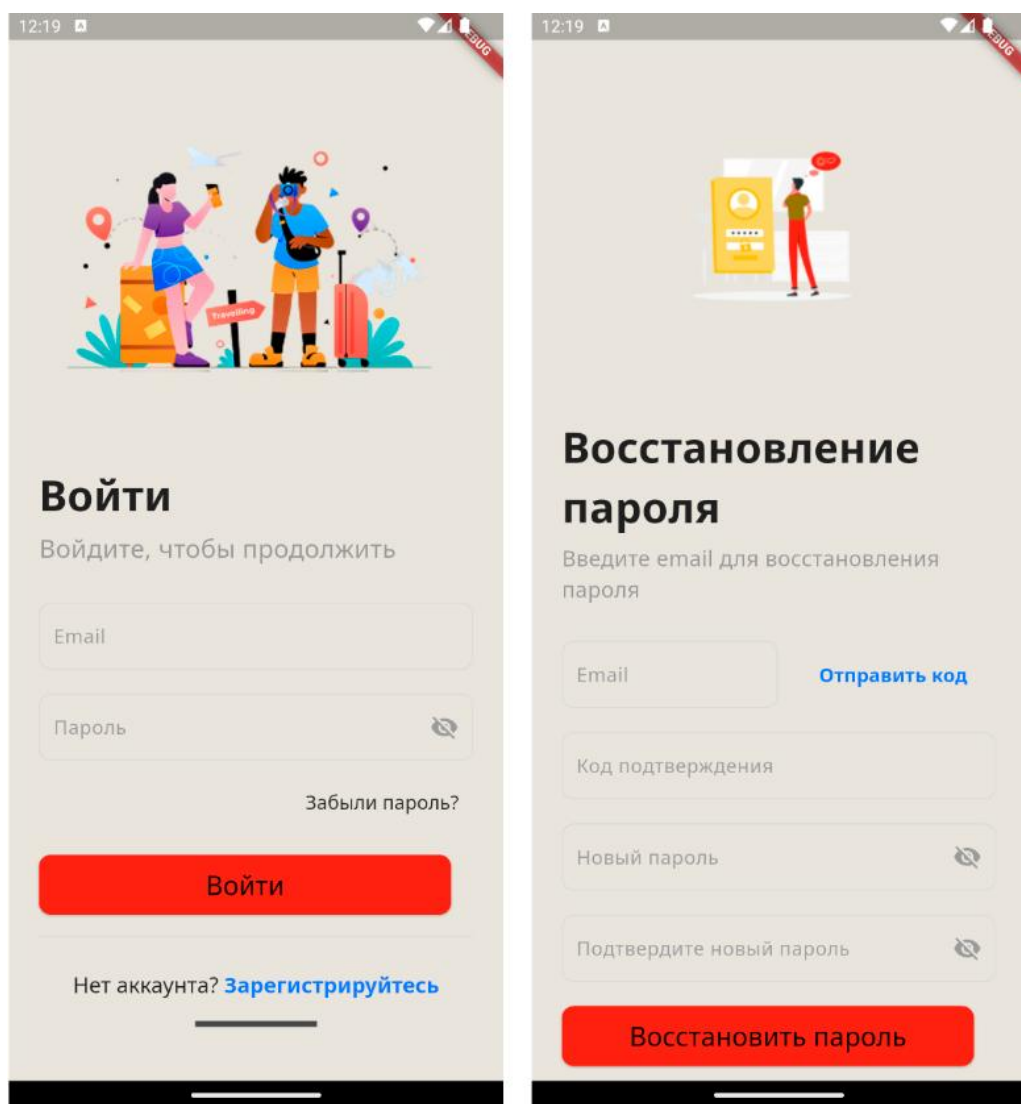


Рисунок 11 — Рорир с предложением сменить пароль

После входа пользователь попадает на главный экран — «Обзор поездок» (см. рисунок 12). Здесь можно выбрать какие будут отображены поездки. Для каждой поездки указаны основные параметры: страна, города, даты, название и статус. В зависимости от статуса поездки (активная, завершённая, запланированная), карточки поездок маркируются разными цветами.

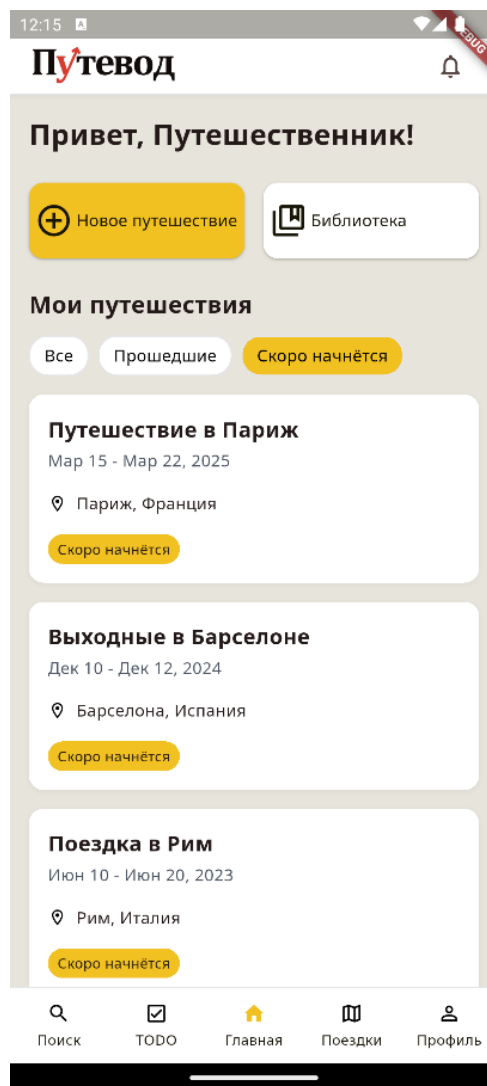


Рисунок 12 — Экран обзора поездок

Также с этого экрана вы можете сразу же перейти к созданию путешествия, в библиотеку экранов или открыть детализацию одного из путешествий.

При переходе во вкладку «Поездки» вы также сможете увидеть все текущие и прошедшие путешествия, и при нажатии на каждое вам будут открываться детали вашей поездки с деталями по каждому дню (см. рисунок 13).

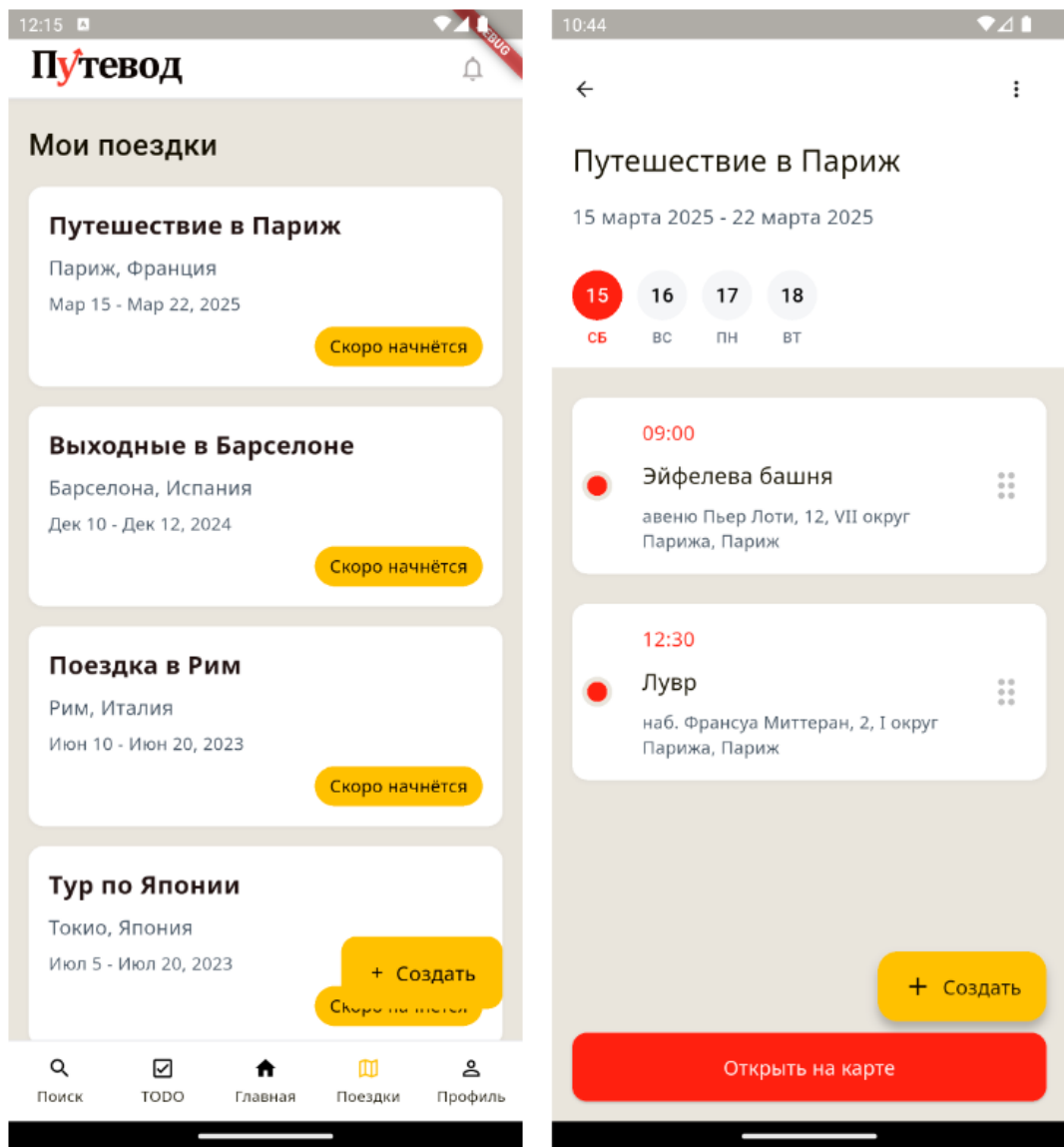


Рисунок 13 — Экраны обзора и деталей поездки

Также вы можете создать поездку, вы нажимаете на специальную кнопку и перед вами открывается пошаговая форма, где пользователь указывает название, даты, описание поездки. А далее появляется пустая поездка, где пользователь может уже в каждый день добавить места события с возможностью точной привязки к времени и координатам или же без них (рисунок 14). В пределах дня вы можете изменять порядок посещения мест с помощью drag-and-drop

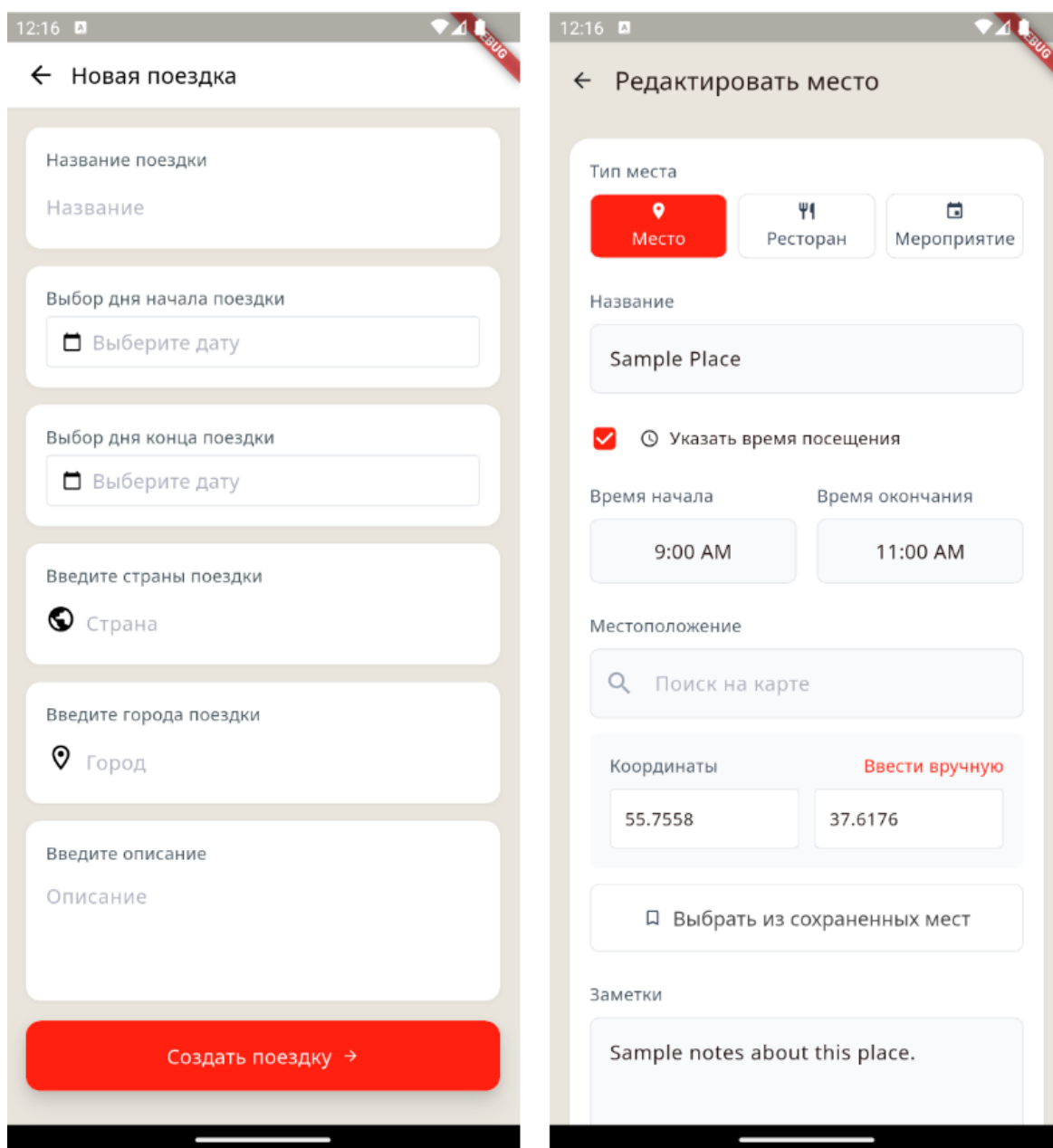


Рисунок 14 — Экраны создания новой поездки и события

Просмотр маршрута и списка событий по дням для лучшей наглядности реализован также с помощью интерактивной карты, где при помощи цифр показана очередность событий, а также при нажатии на любую из иконок вы сможете просматривать подробную информацию о каждом месте (см. рисунок 15).

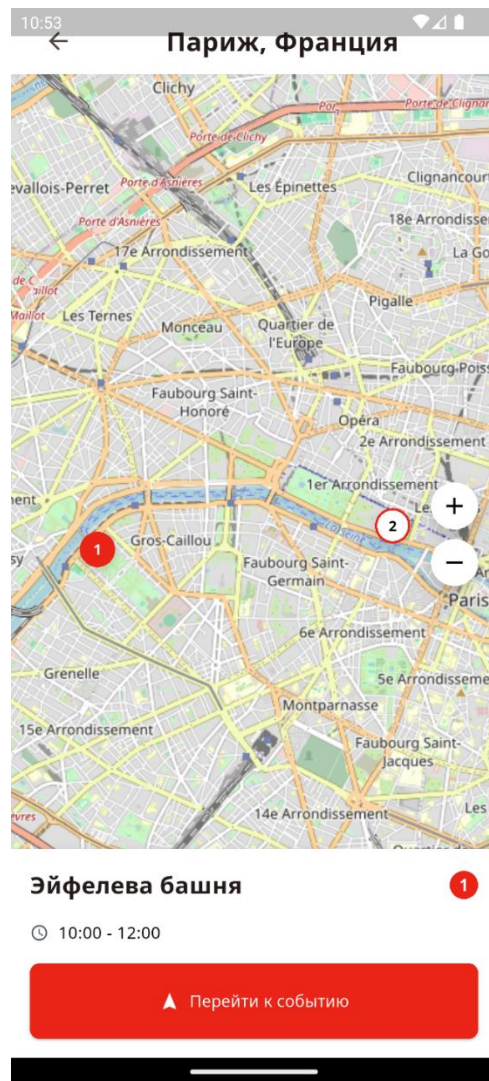


Рисунок 15 — Интерактивная карта маршрута

Во вкладке «TODO» пользователю доступны всего его списки того, что нужно не забыть. Выполненные задачи автоматически перемещаются в конец списка и выделяются менее ярким цветом, также сверху показан прогресс-бар по выполнению todo-листа (см. рисунок 16).

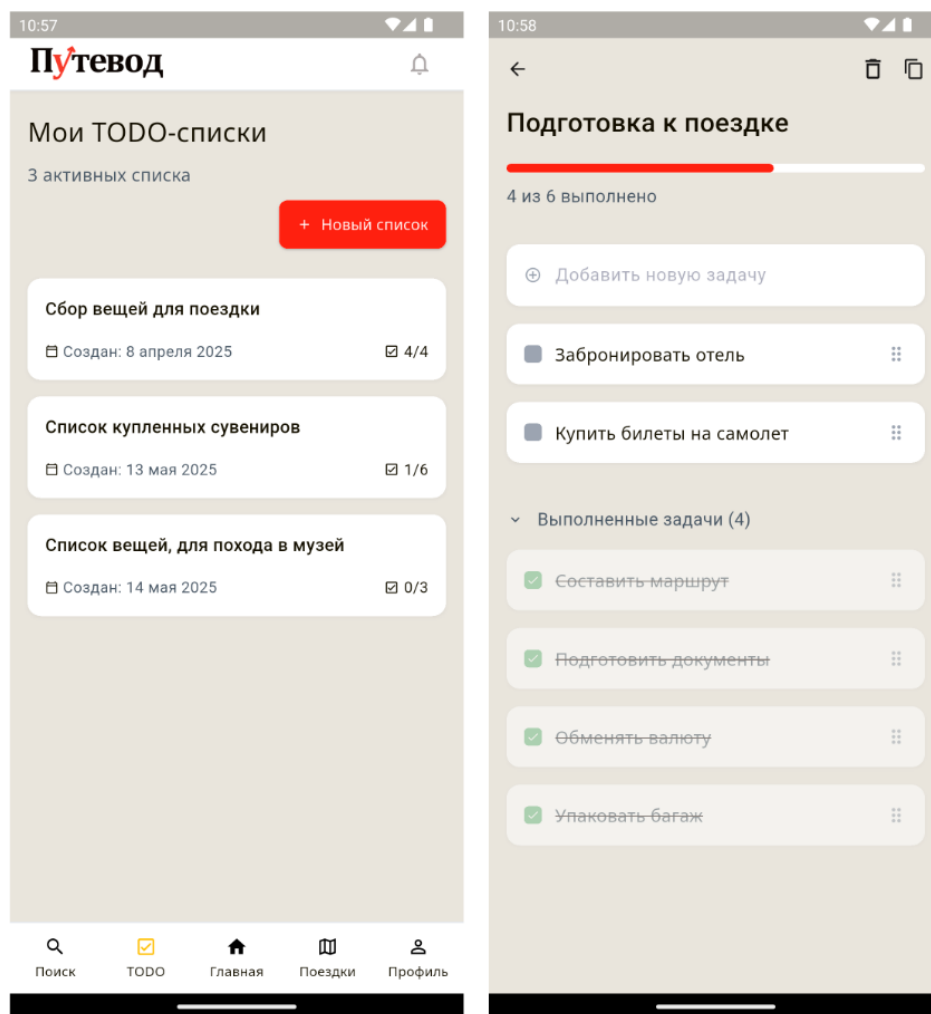


Рисунок 16 — Список задач для подготовки к поездке

Также при создании списка авторизованному пользователю доступна генерация при помощи ИИ, где вы можете сгенерировать список либо на основе поездки или же на основе базовых шаблонов, а также указать информацию через поля и любую недостающие данные написать в поле «дополнительная информация (см. рисунок 17).

11:05

← Генерация списка

Тип поездки

Отпуск

Направление

Paris

Сезон

Лето

Осень

Зима

Весна

Продолжительность

5 дней

Дополнительная информация

Укажите любые дополнительные пожелания

Рисунок 17 — Список задач для подготовки к поездке

При совместном планировании пользователь может пригласить других участников по нику, а изменения маршрута и задач синхронизируются и доступны всем участникам почти сразу же после изменений (см. рисунок 18).

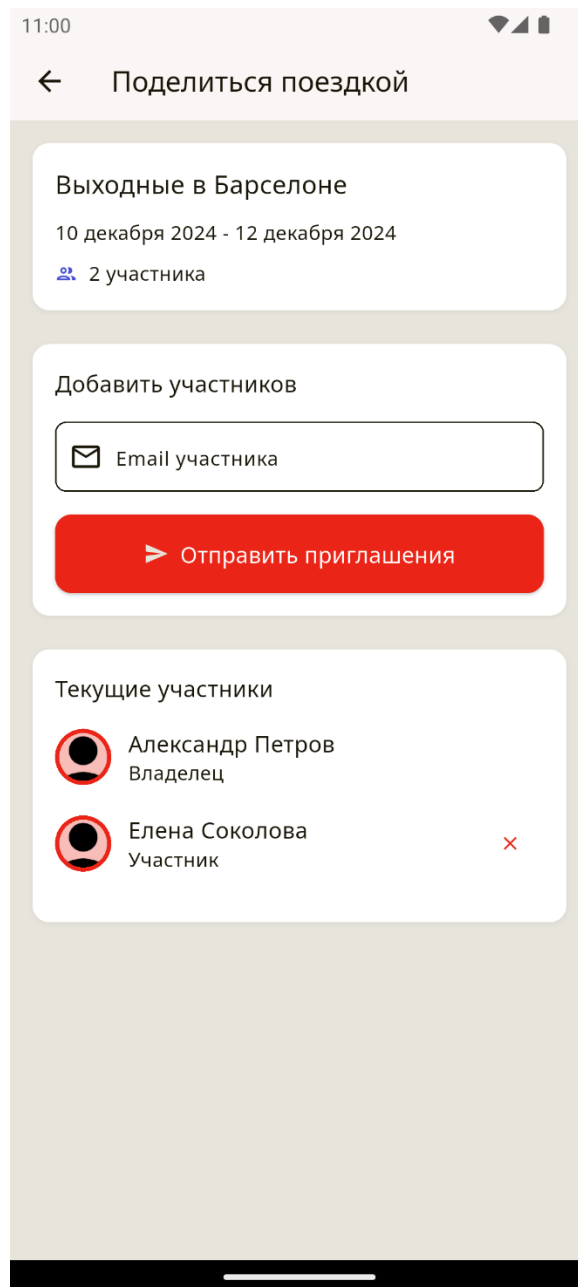


Рисунок 18 — Список задач для подготовки к поездке

Для каждого участника доступен профиль с базовой информацией и возможностью смены никнейма или почты (см. рисунок 19).

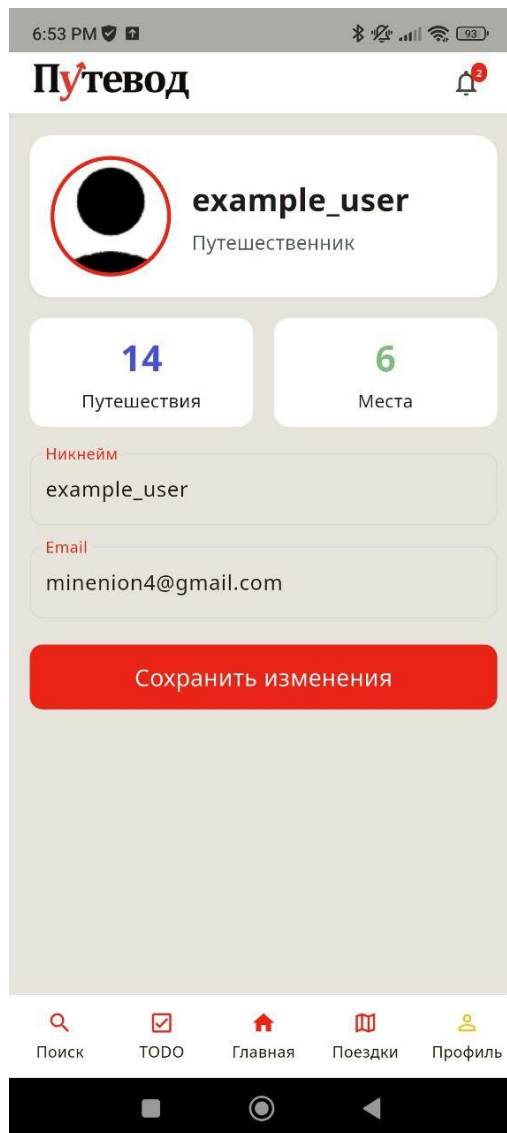


Рисунок 19 — Список задач для подготовки к поездке

Также реализована библиотека маршрутов, где пользователи могут делиться друг с другом поездками и оставлять отзывы о них, сверху вы видите строку поиска, а также на каждом путешествии стоит средняя оценка пользователей (см. рисунок 20)

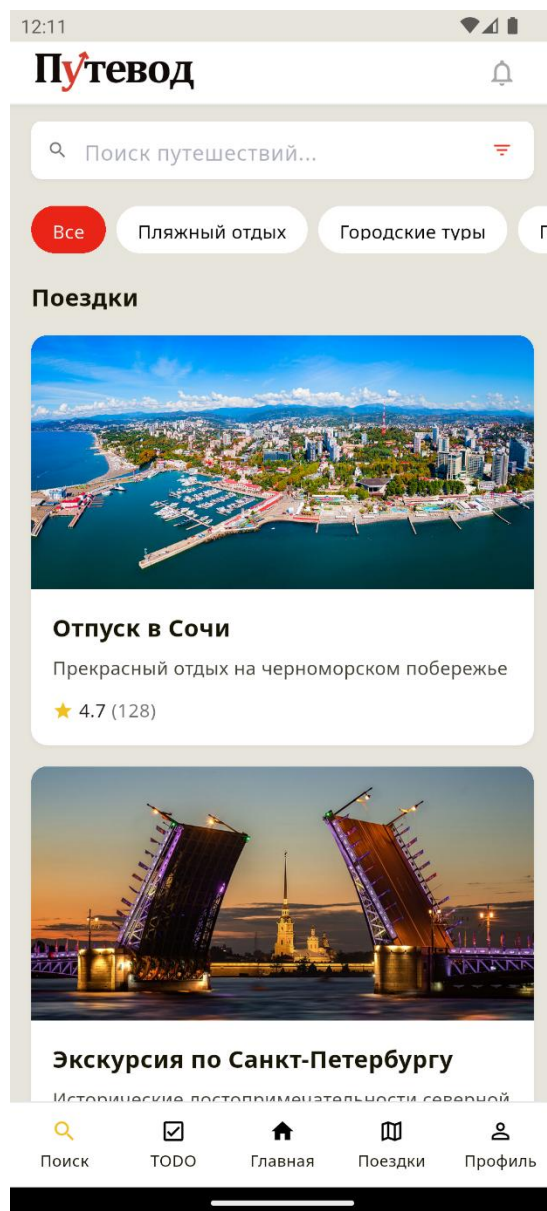


Рисунок 20 — Список задач для подготовки к поездке

А также при переходе в одно из таких путешествий там появляется детальная информация о нём и возможность скопировать его себе в путешествия и использовать в своих целях (см рисунок 21).

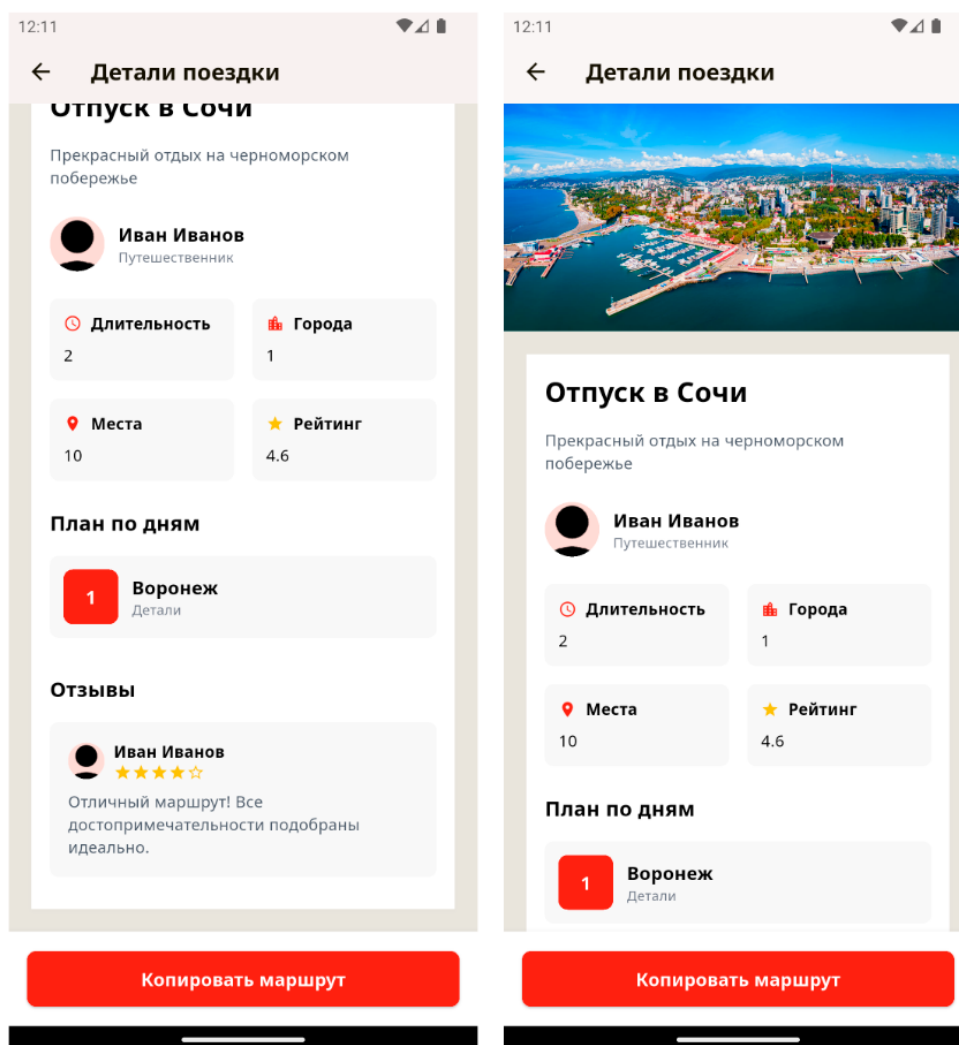


Рисунок 21 — Список задач для подготовки к поездке

3.6.2 Реализация интерфейса сайта для администрирования библиотеки маршрутов

Веб-интерфейс администратора оформлен в едином корпоративном стиле и корректно отображается в актуальных версиях браузеров Google Chrome и Yandex Browser.

Для начала работы администратор должен пройти аутентификацию, после чего получает доступ к панели управления.

На главной странице отображается дашборд с краткой статистикой и превью последних отзывов и маршрутов, ожидающих модерации (см. рисунок 22).

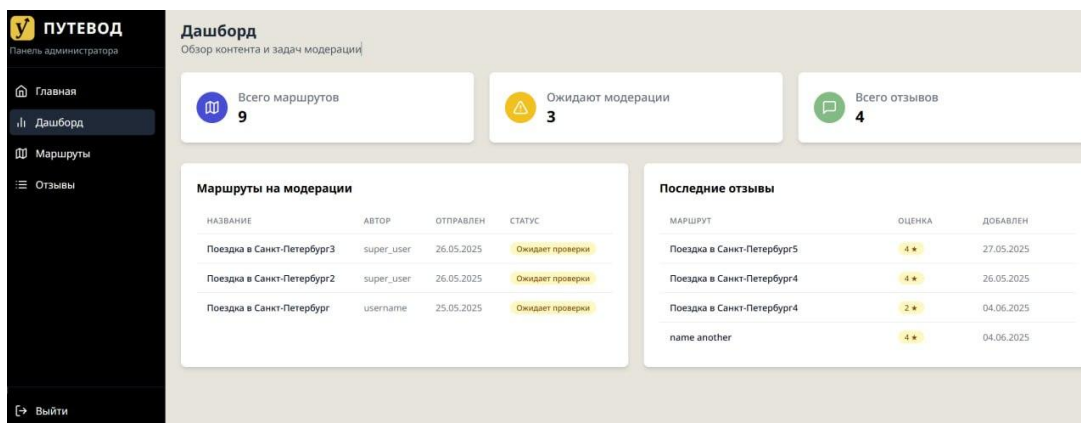


Рисунок 22 — Список задач для подготовки к поездке

При переходе во вкладку «Маршруты» вы сможете увидеть список всех маршрутов и тех, что ожидают модерации для публикации. Вы можете открыть более подробную информацию о маршруте и вам будет дана возможность либо отклонить маршрут, либо же опубликовать его (см. рисунок 23).

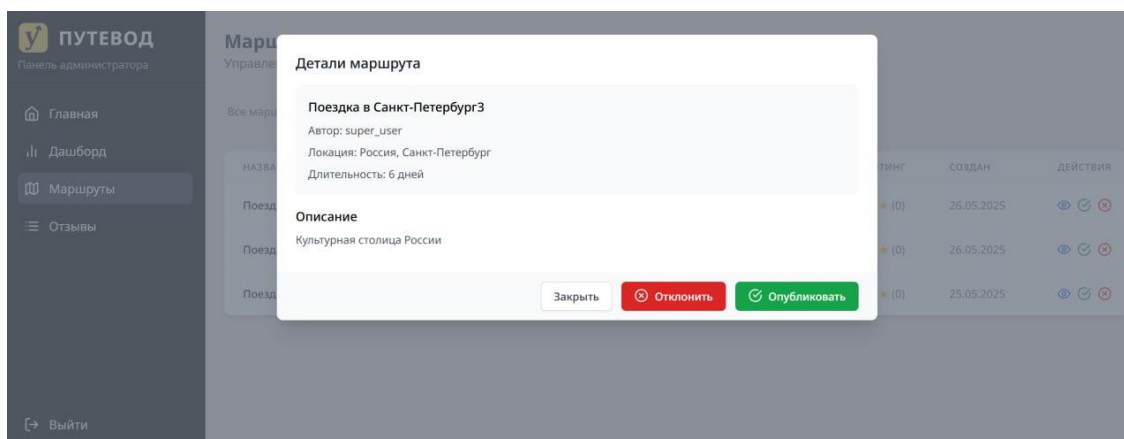


Рисунок 23 — Список задач для подготовки к поездке

4 Тестирование

В ходе тестирования серверной части приложения «Путевод» были использованы ручное тестирование, юнит-тестирование и автоматизированные сценарии. Проверке подверглись все ключевые функции приложения.

Ручное тестирование проводилось с использованием Postman и охватывало базовые сценарии работы REST API — авторизация, регистрация, создание маршрута, добавление задач, работа с библиотекой маршрутов, копирование маршрутов, приглашение соавторов, создание todo-листов. Юнит-тестирование реализовано на стороне серверных компонентов с использованием моков, что позволило покрыть обработчики основных операций и проверить корректность бизнес-логики. Автоматизированные тесты также выполнялись в Postman для проверки как позитивных, так и негативных сценариев: успешное выполнение операций, обработка ошибок, валидация данных и проверка авторизации.

Пользовательский интерфейс мобильного приложения тестировался вручную. Были проверены действия по регистрации, входу, созданию поездки, редактированию маршрута, совместному планированию, работе с интерактивной картой, а также переключение между экранами и реакция интерфейса на разные состояния приложения.

Все операции выполнялись согласно заложенной логике. Ошибки отображались корректно, переходы между экранами осуществлялись без сбоев, а все запросы к API обрабатывались правильно. Элементы пользовательского интерфейса стабильно реагировали на все предусмотренные действия.

Проведённое тестирование подтвердило, что все основные функции приложения «Путевод» работают согласно заявленным требованиям.

5 Аналитика

Для тестирования работоспособности приложения «Путевод» доступ к нему был предоставлен студентам факультета компьютерных наук. В тестировании участвовали 15 пользователей.

На основании собранных данных была построена воронка по сценарию «Создание и совместное редактирование маршрута». Из общего числа пользователей 83,33% успешно создали новую поездку, 41,67% воспользовались функцией добавления соавтора, и только 25% завершили совместное редактирование маршрута (см. рисунок 22). Это демонстрирует, что часть пользователей останавливается на этапе приглашения других участников и взаимодействия с совместным планированием. Можно сделать вывод, что элементы, связанные с приглашением соавторов и организацией совместной работы, требуют доработки в части понятности и доступности в пользовательском интерфейсе.

Заключение

В ходе реализации данного проекта была создана основная функциональность мобильного приложения. Разработанное мобильное приложение «Путевод» соответствует предъявленным требованиям, а именно:

- обеспечивает возможность создания, редактирования и управления маршрутами поездок с почасовой детализацией;
- позволяет формировать и вести индивидуальные TODO-списки для подготовки к путешествию, а также создавать их на основе ИИ;
- реализует совместное планирование маршрутов с другими пользователями;
- предоставляет возможность делиться поездками с другими пользователями приложения.

В результате проведённого тестирования было установлено, что приложение успешно прошло все предусмотренные сценарии использования, продемонстрировав стабильную работу и соответствие заявленным функциональным требованиям.

Список использованных источников

1. BusinesStat. Внутренний туристический поток в России в 2024 г.: аналитический обзор [Электронный ресурс]. – URL: https://www.tadviser.ru/index.php/Статья:Туризм_в_России (дата обращения: 04.06.2025).
2. Фонд «НАФИ». Тренд на самостоятельные путешествия заметно усилился за пять лет [Электронный ресурс]. – URL: <https://nafi.ru/analytics/trend-na-samostoyatelnye-puteshestviya-zametno-usililsya-za-pyat-let/> (дата обращения: 04.06.2025).
3. ВЦИОМ. Автотуризм-2024: результаты всероссийского опроса [Электронный ресурс]. – URL: <https://wciom.ru/analytical-reviews/analiticheskii-obzor/avtoturizm-2024> (дата обращения: 02.06.2025).
4. Flutter. Документация разработчика [Электронный ресурс]. – URL: <https://docs.flutter.dev/> (дата обращения: 05.06.2025).
5. Spring Boot. Справочник (Reference Guide) [Электронный ресурс]. – URL: <https://docs.spring.io/spring-boot/documentation.html> (дата обращения: 05.06.2025).
6. Docker Docs. Swarm mode overview [Электронный ресурс]. – URL: <https://docs.docker.com/engine/swarm/> (дата обращения: 05.06.2025).
7. OpenTripMap API. Описание методов [Электронный ресурс]. – URL: <https://opentripmap.io/docs> (дата обращения: 05.06.2025).
8. Unsplash. Image API Documentation [Электронный ресурс]. – URL: <https://unsplash.com/documentation> (дата обращения: 05.06.2025).

ПРИЛОЖЕНИЕ А

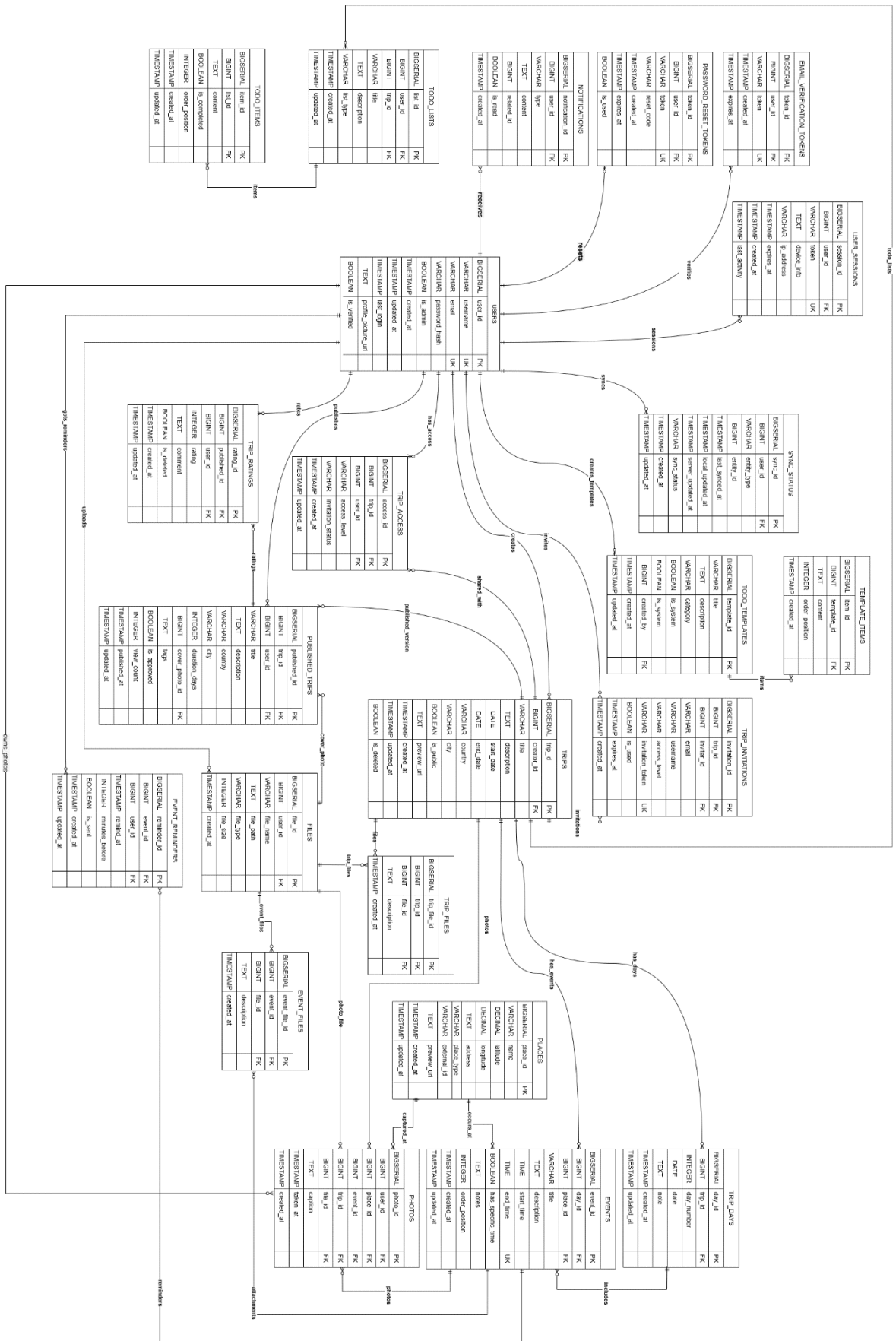


Рисунок А.1 — ER-диаграмма базы данных

ПРИЛОЖЕНИЕ Б

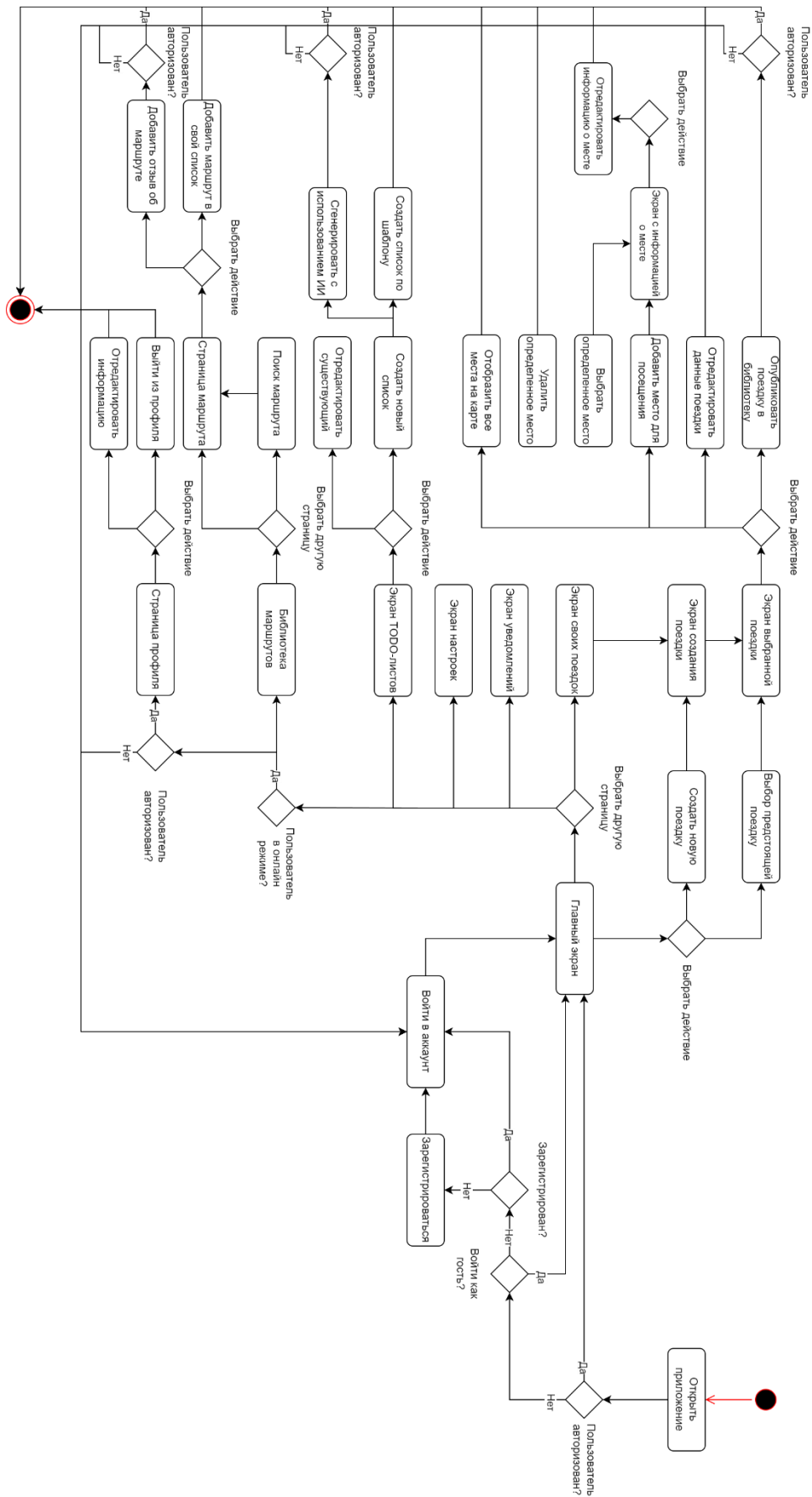


Рисунок Б.1 — Диаграмма активности пользователя

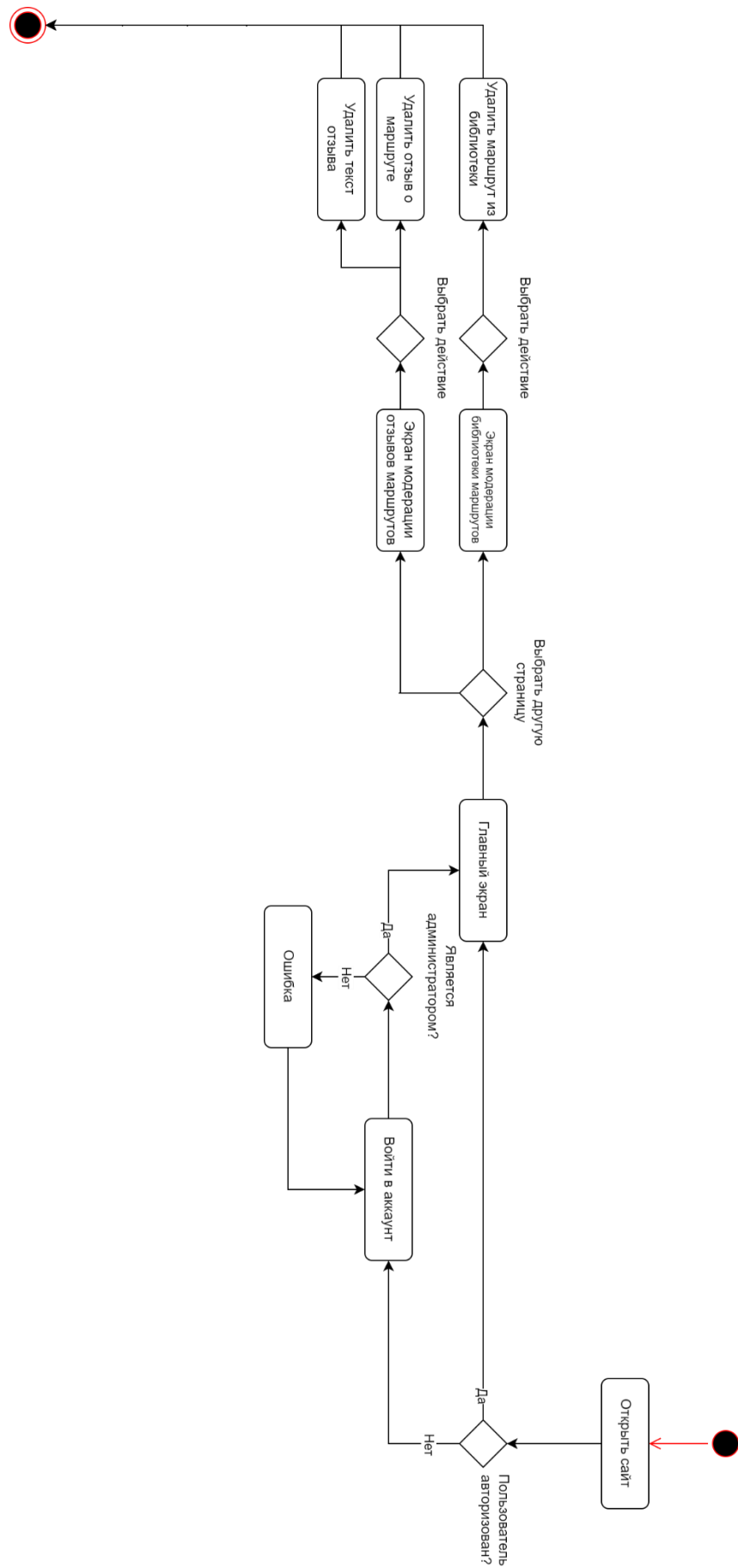


Рисунок Б.2 — Диаграмма активности администратора